规格驱动开发：AI 时代软件开发新范式

李 祎[1]

(1．广州软件学院软件与人工智能学院，广东广州 510990)

摘 要：以自然语言驱动代码生成的“Vibe Coding”范式在提升原型产出速度的同时，带来了正确性难以保障、意图难以追溯与验证闭环缺失等工程挑战。本文提出规格驱动开发（Specification-Driven Development, SDD）方法论，将可执行、可验证的规格置于开发流程核心，使需求、实现与测试在统一语义空间中形成闭环。本文首先分析 Vibe Coding 的工程风险与工具生态，继而给出 SDD 的流程框架，最后通过 Amazon Kiro、GitHub Spec Kit 与 OpenSpec 三个项目的案例分析展示 SDD 的实践路径。

关键词：规格驱动开发；Vibe Coding；AI 辅助编程；软件工程方法论

Specification-Driven Development: A New Paradigm for Software Development in the AI Era

LI Yi[1]

(1. School of Software and Artificial Intelligence, Guangzhou University of Software, Guangzhou Guangdong 510990, China)

Abstract: The “Vibe Coding” paradigm—driving code generation through natural language—accelerates prototyping but introduces challenges in correctness assurance, intent traceability, and verification closure. This paper proposes Specification-Driven Development (SDD), a methodology that places executable, verifiable specifications at the center of the development process so that requirements, implementation, and testing form a closed loop within a unified semantic space. We analyze the engineering risks of Vibe Coding, present the SDD process framework, and demonstrate its practical benefits through case studies of three projects: Amazon Kiro, GitHub Spec Kit, and OpenSpec.

Keywords: specification-driven development; Vibe Coding; AI-assisted programming; software engineering methodology

1 引言

过去十年，人工智能在软件开发领域的角色经历了从辅助到深度参与的演进：从静态分析与代码补全，到仓库级上下文中的对话式生成，再到能够主动规划、执行与回收反馈的代理式编程（agentic coding）范式[1]。以“Vibe Coding”为代表的自然语言驱动编程工作流正是这一演进的集中体现——开发者以自然语言描述目标与约束，由模型生成代码、补充测试与文档，并通过运行结果或评审意见不断迭代[2]。

这种工作方式显著降低了“从想法到可运行系统”的门槛，但也放大了传统软件工程中的若干痛点。首先，在**正确性与一致性**方面，模型生成的内容可能遗漏边界条件、误解需求或引入不一致的实现。其次，在**可维护性**方面，快速生成的代码若缺少统一的架构约束与风格规范，长期演进成本将显著上升。第三，在**责任边界**方面，当错误来源可能是提示词、规格、模型推断或集成

基金项目：广东省普通高校青年创新人才类项目“基于生成式人工智能的测试用例生成技术研究”（项目编号：2023KQNCX118）；广东省高等教育教学改革项目“AI 赋能高校程序设计类课程教学改革与研究”（项目编号：粤教高函 [2026]4 号-1254）；2025 年度广州软件学院质量工程建设项目“AI 大模型应用课程群教研室”（项目编号：JYS202503）

作者简介：李祎，男，硕士，讲师、系统架构设计师。研究方向：软件架构设计、代码智能体。

工具链中的任一环节时，定位与问责变得复杂。因此，AI 时代软件工程的核心挑战不再只是“写得更快”，而是“如何将意图转化为可验证的约束，并在迭代中持续保持一致”。

值得注意的是，“AI 必然提升开发效率”的直觉并非总是成立。不同任务类型、上下文复杂度与工具链成熟度会显著影响产出质量与调试成本。近期有实证研究在部分真实开发任务中观察到效率下降的现象[3]，这提示我们需要将工程方法论（规格、验证、质量关卡）与工具能力一并设计，而非仅仅优化提示词。

基于上述观察，本文提出规格驱动开发（Specification-Driven Development, SDD）方法论。其核心主张是：在编码之前与编码过程中，优先将“系统应该做什么、不能做什么、如何验证做对了”以结构化且可检验的形式表达出来，并以这些规格作为生成、评审、测试与演进的共同依据。这里的“规格”是多层次的，涵盖业务需求、接口契约、行为状态、性质不变量与非功能约束等维度，具体内容将在第 4 节详述。

SDD 在 AI 时代的优势体现在四个方面：其一，**对齐**——将提示词中的隐含意图固化为团队共享、可评审的规格，减少误解与反复沟通；其二，**可追溯**——需求、规格、实现与测试形成完整链路，便于回溯变更影响；其三，**可验证**——通过测试、契约检查与性质验证，将“看起来正确”转化为“有证据支持的正确”；其四，**可迭代**——规格先行使分阶段交付更为自然，可先交付最小可验证切片，再逐步扩展规格与实现。

本文其余部分的组织如下：第 2 节梳理相关工作；第 3 节分析 Vibe Coding 的工程风险与工具生态，阐明 SDD 的提出动机；第 4 节给出 SDD 的方法论框架；第 5 节通过三个项目案例展示 SDD 的工程化实践；第 6 节总结全文并讨论未来方向。

2 相关工作

SDD 并非凭空提出，而是将需求工程、契约式设计与可验证性实践在 AI 辅助开发的新语境下重新组合。事实上，Ostroff 等人早在 2004 年便提出了“敏捷规格驱动开发”的思路，将形式化规格与敏捷迭代相结合[4]。本文在此基础上，面向 AI 代理式编程的新场景重新审视规格的角色与流程设计。本节梳理与之密切相关的研究基础。

测试驱动开发与行为驱动开发。测试驱动开发（TDD）由 Beck[5] 提出，其核心思想是以可运行测试驱动实现过程，通过“红–绿–重构”循环减少回归缺陷与返工成本。行为驱动开发（BDD）进一步将验收标准以自然语言格式（如 Given-When-Then）表达，使业务方与开发方在同一语义层上对齐。SDD 继承了“先定义预期行为、再实现”的思路，但将规格的范围从测试用例扩展到涵盖契约、行为与性质的多层次描述体系。

属性测试与基于性质的验证。Claessen 与 Hughes 提出的 QuickCheck[6] 开创了属性测试（property-based testing）方向：以性质（property）描述程序不变量，由框架自动生成大量随机样本寻找反例，从而以较低的人工成本覆盖更多边界情况。SDD 将属性测试纳入验证层的核心组件，用于持续检验规格中声明的不变量与安全约束。

形式化方法。以 Lamport 的 TLA$^+$[7] 为代表的形式化规格语言，能够以数学精确的方式描述系统行为，并通过模型检查（model checking）验证并发与分布式系统的关键性质。尽管完全形式化验证的成本较高，其“将行为与性质显式化”的思想为 SDD 中的行为规格与不变量声明提供了理论根基。

AI 辅助代码生成及其风险。大语言模型（如 OpenAI Codex[8]）展示了从自然语言描述生成可运行代码的能力，但相关研究也揭示了其局限性。Pearce 等人[9] 的实证分析表明，AI 生成的代码可能引入安全缺陷（如输入校验缺失、注入漏洞等），强调了将安全约束纳入规格并以自动化质量关卡强制执行的必要性。SDD 正是回应这一挑战的方法论尝试：通过将意图显式化为可检验规格，为 AI 生成的代码提供结构化的评判标准与验证机制。

3 Vibe Coding：动机与挑战

本节分析 Vibe Coding 的工作方式及其暴露的工程风险，阐明 SDD 的提出动机；随后梳理当前工具生态与典型应用场景，指出规格先行对于释放工具潜力的必要性。

3.1 工作方式与工程风险

Vibe Coding 通常指一种以“对话–生成–反馈”为核心的开发方式[10,11]：开发者通过自然语言或半结构化指令表达意图，由模型在项目上下文中生成或修改代码，再经运行结果、测试与评审形成反馈，驱动下一轮迭代。与传统开发相比，Vibe Coding 在**表达介质**（对话替代设计文档）、**迭代步长**（快速产出可运行雏形）和**上下文依赖**（输出质量取决于提示词与约束的明确程度）三个层面带来了显著变化。

然而，当 Vibe Coding 主要依赖对话中的隐含意图而缺少可检验规格时，工程代价便以调试与返工的形式出现。在**语义偏移**方面，模型将自然语言解释为实现细节时可能产生偏差，后续修正若缺少规格基线，容易在多轮迭代中反复震荡[8]。在**安全风险**方面，自动生成的代码可能引入输入校验缺失、命令注入等不安全模式，需要将安全约束纳入规格并以自动化关卡强制执行[9]。在**可复现性**方面，对话过程难以复盘，提示词与上下文随时间漂移，应以版本化规格与可运行验证来固化证据链。上述风险的共同根源在于意图未被显式固化为可检验的规格——这正是 SDD 试图解决的核心问题。

3.2 工具生态与应用现状

当前，Vibe Coding 的工具生态已初具规模：Cursor[12] 提供 IDE 集成式 AI 编程体验，Claude Code[13] 以命令行与代理工作流见长，Codex[14] 面向代码生成与任务执行。这些工具各有侧重，但共同依赖于开发者提供的上下文质量，从工具层面印证了规格先行的必要性。在工程实践中，Vibe Coding 已广泛应用于脚手架生成、增量功能开发、遗留系统重构、跨语言迁移、测试补齐与文档自动化等场景[2]，其共同特征是：模型擅长快速产出候选实现，而工程成功与否取决于是否有清晰的规格与验证闭环来约束产出质量。

4 规格驱动开发方法论

本节给出规格驱动开发的流程框架，并阐述每一阶段如何与 AI 工具配合形成“规格–实现–验证”的闭环。如图 1 所示，SDD 的核心流程由四个阶段组成：问题定义、可检验规格编写、生成与实现、验证与质量关卡，四者构成持续迭代的闭环。

4.1 需求收集与问题定义

在 SDD 中，需求收集的重点不是一次性穷举所有细节，而是先明确可讨论的边界，使后续规格能够被验证与迭代。推荐以“问题定义卡”（Problem Statement）为起点，涵盖目标与非目标、核心用例（输入、触发条件、预期输出与异常分支）、约束条件、风险清单以及可直接转化为测试的验收标准（如 Given-When-Then 格式）。AI 工具可用于快速补全边界条件与提出反例，但最终的验收标准必须由团队落实为文字与示例。

4.2 规格编写

规格编写阶段的目标是将意图转换为结构化、可检查、可生成测试的描述。一个可操作的做法是将规格划分为三个层次。**契约层（Contract）**定义接口与数据结构，包括端点路径、字段含义、必填与可选约束、取值范围、版本与兼容策略等。**行为层（Behavior）**描述关键流程的前置条件、后置条件与状态转移，并对并发、幂等、重试与补偿的语义给出明确约束。**性质层（Properties）**声明必须长期保持的断言与不变量，以及安全与合规约束。

契约层推荐尽量采用可被工具消费的标准格式：例如以 OpenAPI[15] 统一表达 HTTP API，以 JSON Schema[16] 表达数据结构与约束（也可作为 OpenAPI 3.1 的 Schema 基础），从而便于后续的代码生成、lint 检查、差异检测与契约测试联动。同时，规格需要包含可执行的反例与边界描述——例如输入为空、重复请求、权限不足或外部依赖超时等情况下系统应如何响应——并对错误处理统一约定错误分类、错误码语义、可恢复策略（重试/降级/告警）与观测点（日志/指标/追踪）。

在 AI 辅助下，规格编写可以先由模型根据需求草拟，再由团队进行三类审查：可验证性（能否写成测试）、完备性（是否覆盖主要边界）与一致性（不同模块间是否自洽）。

表 1: Vibe Coding 与 SDD 的核心对比

维度	Vibe Coding（弱规格）	规格驱动开发（SDD）
意图表达	沉淀在对话与提示词中，易随上下文漂移	以可检验规格固化（契约/行为/性质），可评审、可版本化
迭代方式	生成 → 运行 → 修补，依赖人工判断	规格 → 生成 → 自动化验证，失败以反例回写规格
质量证据	以“能跑通”为主，边界覆盖不稳定	测试、契约检查与性质验证形成证据链
风险控制	错误晚期暴露，定位依赖个人经验	风险前移至规格层，CI 质量关卡强制一致性
协作与交接	高度依赖个体提示词与心智模型	共享规格与验收标准降低沟通与交接成本

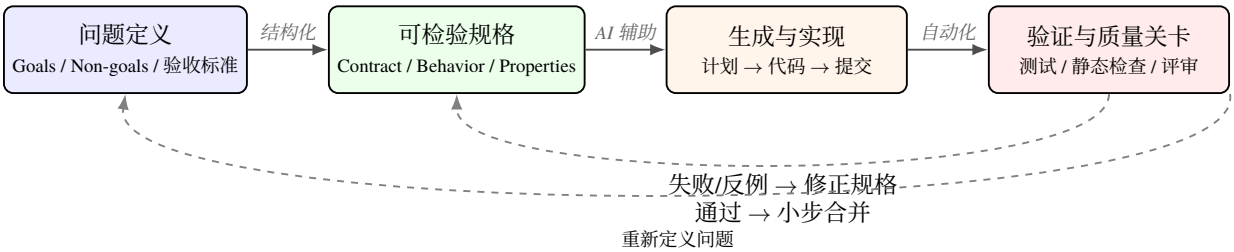

图 1: 规格驱动开发（SDD）的闭环流程

4.3 AI 辅助代码生成

当规格具备一定结构后，代码生成应遵循“先薄片、后扩展”的原则：优先生成最小可验证切片（thin-slice），让端到端路径跑通，并尽早建立验证闭环。

具体而言，生成过程可按以下步骤组织。首先，根据契约与行为规格进行**模块划分**，拆分 API 层、领域层、数据访问层与适配器层的边界，明确每个模块的输入输出与职责。其次，在**生成策略**上要求模型“先输出计划再输出代码”，并在代码中保留与规格条款的对应标注。第三，在**约束优先**原则下，将规格中的不变量、错误处理与观测点作为硬约束——当模型无法满足时，应回到规格层讨论而非在实现层默默妥协。第四，遵循**小步合并**策略，确保每次变更都可编译、可运行、可测试，对大幅重构则拆分为多个可回滚的小提交。

在该阶段，AI 的价值在于将实现成本从“逐行编码”转移到“规格校验与审查”，而 SDD 的价值在于为审查提供了明确的裁判标准——是否满足规格、是否有测试证据支撑。

4.4 测试与验证

SDD 的“驱动”二字集中体现在验证机制上：规格不是写完就搁置的文档，而是需要持续被测试与检查工具消费的活跃资产。

推荐的验证层次包括四个维度。**单元测试**覆盖核心业务规则与边界条件，确保局部逻辑稳定。**契约测试**围绕接口契约验证请求/响应结构、错误码语义与向后兼容策略，典型实践如消费者驱动的契约测试[17]。**属性测试**将不变量写成性质（properties），用随机数据或生成器探索边界与反例[6]。**端到端测试**验证关键用户路径，并对外部依赖进行可控的桩与模拟。

为将验证嵌入工程常态，建议在持续集成中设置“规格一致性关卡”（quality gate）：合并前必须通过自动化测试；规格变更时要求同步更新相关测试与文档；对关键性质（如权限、幂等与资金安全）设置更严格的检查与审计。在 Vibe Coding 场景下，这些质量关卡能够显著降低“模型输出看起来合理但实际不可用”的风险，将迭代的主要成本压缩到可控范围内。

5 规格驱动开发案例

本节聚焦三个具有代表性的规格驱动开发项目——Amazon Kiro、GitHub Spec Kit 与 OpenSpec——展示 SDD 如何在实际工具中落地。三者分别从不同角度诠释了" 规格先行" 理念: Kiro 以三阶段结构化工作流与 EARS 需求表示法将模糊意图转化为可追踪的规格工件，Spec Kit 以层级式工件体系与 CLI 工具链支撑从规格到实现的端到端生成流程, OpenSpec 以流动式变更驱动框架实现规格的持续迭代与动态一致。[15,16,18,20–22]

5.1 Amazon Kiro：结构化规格工作流

Amazon Kiro[18,19] 是亚马逊推出的规格驱动开发工具，其核心理念是通过结构化的规格工作流将模糊的产品构想转化为可追踪、可验证的实现计划。Kiro 围绕" 需求–设计–实现" 三阶段工作流组织开发过程，每个阶段对应一个独立的规格文件。

Kiro 为每个功能规格生成三类核心工件：**需求文档**（`requirements.md`）采用 EARS（Easy Approach to Requirements Syntax）表示法, 以"WHEN [条件/事件] THE SYSTEM SHALL [预期行为]" 的结构化句式捕获用户故事与验收标准，确保每条需求具备可测试性与可追溯性；**设计文档**（`design.md`）记录系统架构、序列图、数据模型、接口设计与错误处理策略等技术决策；**任务清单**（`tasks.md`）将设计分解为离散的、可追踪的实现任务，并提供实时状态更新的执行界面。

从方法论角度看，Kiro 的贡献在于将 EARS 表示法引入 AI 辅助开发场景：该表示法通过固定句式消除自然语言需求的歧义性，使每条需求天然具备可验证性，同时为 AI 代理提供了结构化的输入格式。这种" 先结构化需求、再设计架构、最后分解任务" 的渐进式工作流，为产品与工程团队之间的协作提供了清晰的对齐点，有效降低了从意图到实现的语义损耗。

5.2 GitHub Spec Kit

GitHub Spec Kit[20,21] 是由 GitHub 开源的规格驱动开发工具包，其核心理念是将规格从“沟通文档”转变为“可执行工件”——规格直接作为 AI 代理的输入，驱动实现的自动生成。

Spec Kit 定义了五类层层递进的规格工件：**宪章**（治理原则）、**规格**（结构化需求）、**计划**（技术策略）、**任务**（可执行分解）与**实现**（代码产出），形成从抽象意图到具体实现的可追溯链路。在工具链层面，Spec Kit 提供命令行工具 `specify-cli`，并通过斜杠命令机制（如 `/speckit.specify`、`/speckit.plan`、`/speckit.implement` 等）与 Claude Code、Cursor、GitHub Copilot 等十余种 AI 编码代理集成[21]。

Spec Kit 支持从零构建（greenfield）、并行探索（同一规格下对比不同方案）与增量演进（brownfield）三种典型场景。其方法论启示在于：与其依赖临时的高质量提示词，不如将结构性知识固化为可复用的规格模板与工件体系，融入持续集成流程以形成可持续的工程能力。

5.3 OpenSpec：流动式规格工件化框架

OpenSpec[22,23] 是一个面向 AI 编码助手的轻量级规格驱动开发框架，其设计哲学强调" 流动而非刚性、迭代而非瀑布、简洁而非繁重"。与 Spec Kit 的层级式工件体系不同, OpenSpec 以" 变更包（change）" 为基本单位组织工作：每个功能或修复对应一个独立目录, 包含提案（`proposal.md`）、需求与场景（`specs/`）、技术设计（`design.md`）与任务清单（`tasks.md`）四类工件，完成后通过归档机制将证据链固化为带时间戳的永久记录。

OpenSpec 采用基于动作的命令模式替代传统的阶段锁定流程[23]，核心命令包括：`/opsx:explore`（探索需求与方案）、`/opsx:new`（创建变更目录）、`/opsx:ff`（一次性生成全部规划工件）、`/opsx:apply`（按任务清单执行实现）、`/opsx:verify`（从完备性、正确性与一致性三个维度验证实现）以及 `/opsx:archive`（归档并同步规格库）。开发者可在任意阶段自由切换，不受严格的阶段顺序约束。该框架支持 GitHub Copilot、Claude、GPT 等 20 余种 AI 助手，并通过增量规格（delta specs）机制捕获实现过程中的规格修正，使规格与实现保持动态一致。

从方法论角度看，OpenSpec 将 AI 的输出从" 直接生成代码" 前移为" 生成可评审的规格与任务结构"，在多人协作中更易落实责任边界。其规格工件还可与 Spectral[24] lint 检查、OpenAPI Generator[25] 代码骨架生成、Pact[17] 契约测试等开放工具链对接, 形成自动化的规格一致性关卡。

6 总结与展望

本文围绕 AI 辅助编程时代的工程挑战，提出了规格驱动开发（SDD）方法论，并从概念框架、流程设计与开源实践三个层面进行了阐述。

SDD 的核心贡献在于提供了一条将 Vibe Coding 的效率优势与传统软件工程的质量保障相结合的路径：通过将意图固化为可检验的多层次规格，使需求对齐、代码生成与质量验证在统一的语义框架下闭环运作，从而降低 AI 生成代码的返工成本，提升协作效率与可追溯性。Amazon Kiro、GitHub Spec Kit 与 OpenSpec 的案例分析表明，规格工件化与工具链集成已具备工程可行性，能够在实际开发流程中落地。

然而，SDD 的广泛应用仍面临若干开放性挑战。在**规格表达的标准化**方面，当前不同层次的规格（业务需求、接口契约、行为约束、系统性质）尚缺少统一的描述标准与互操作框架，这限制了工具链的通用性与可组合性。在**复杂系统规格的可维护性**方面，随着系统规模增长，规格自身的一致性管理与演进成本可能成为新的瓶颈。在**模型可靠性**方面，大语言模型的“幻觉”问题尚未根本解决，AI 生成的规格草案与实现代码仍需人工审查与验证机制兜底。在**度量与责任**方面，如何量化规格覆盖率、规格与实现的一致性，并在团队中建立清晰的责任归属机制，也有待进一步探索。

未来研究可在以下方向展开：构建跨层次规格的形式化关联模型，探索规格自动演进与一致性检查的工具支持，开展 SDD 在不同规模与领域项目中的实证评估，以及研究人机协作模式下规格编写与审查的最优分工策略。

参考文献

[1] Haolin Jin, Linghan Huang, Haipeng Cai, et al. *From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future*. arXiv:2408.02479, 2025.

[2] Markus Borg, Elizabeth Bjarnason, Fabian Hedin. *Vibe Coding and the New Prototyping Playbook*. *IEEE Software*, 42(6):12–16, 2025.

[3] METR. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. METR blog, 2025. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[4] Jonathan S. Ostroff, David Makalsky, Richard F. Paige. *Agile Specification-Driven Development*. In *Proc. XP/Agile Universe*, LNCS 3092, pp. 104–112, Springer, 2004.

[5] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[6] Koen Claessen, John Hughes. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*. In *Proc. ICFP*, pp. 268–279, 2000.

[7] Leslie Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021.

[9] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, et al. *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. In *Proc. IEEE S&P*, pp. 754–768, 2022.

[10] Kyle Orland. “Vibe coding” is all the rage. It's also a terrible idea. Ars Technica, 2025. https://arstechnica.com/information-technology/2025/02/vibe-coding-is-all-the-rage-its-also-a-terrible-idea/

[11] Nikolaos Malamas, Emmanouil Tsardoulias, Konstantinos Panayiotou, et al. *Toward Efficient Vibe Coding: An LLM-based Agent for Low-Code Software Development*. *Journal of Computer Languages*, 85:101367, 2025.

[12] Cursor. *Cursor Documentation*. https://docs.cursor.com/

[13] Anthropic. *Claude Code Overview*. https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview

[14] OpenAI. *Codex*. https://platform.openai.com/docs/codex

[15] OpenAPI Initiative. *OpenAPI Specification*. https://spec.openapis.org/oas/latest.html

[16] JSON Schema. *JSON Schema Specifications and Documentation*. https://json-schema.org/

[17] Pact Foundation. *Pact: Consumer-Driven Contract Testing*. https://docs.pact.io/

[18] Amazon. *Kiro Specs Documentation*. https://kiro.dev/docs/specs/

[19] Amazon. *Kiro Specs Concepts*. https://kiro.dev/docs/specs/concepts/

[20] GitHub. *Spec Kit Documentation*. https://github.github.com/spec-kit/

[21] GitHub. *github/spec-kit: An open source toolkit for Spec-Driven Development*. https://github.com/github/spec-kit

[22] Fission AI. *Fission-AI/OpenSpec*. https://github.com/Fission-AI/OpenSpec

[23] Fission AI. *OpenSpec Workflows*. https://github.com/Fission-AI/OpenSpec/blob/main/docs/workflows.md

[24] Stoplight. *Spectral: Open-Source API Style Guide Enforcer*. https://github.com/stoplightio/spectral

[25] OpenAPITools. *OpenAPI Generator*. https://github.com/OpenAPITools/openapi-generator